Computer Architecture 2 | | HW#2
SIMD Vectorization

	Vector-Vector Multiplication		
Input-Size	**Scalar**	**Vector**	**Improvement(%)**
128	0.000001	0.000001	1
256	0.000001	0.000001	1
512	0.000002	0.000001	2
	Matrix-Vector Multiplication		
Input-Size	**Scalar**	**Vector**	**Improvement(%)**
128	0.000040	0.000015	2.6667
256	0.000124	0.000077	1.6104
512	0.000518	0.000220	2.3545
	Matrix-Matrix Multiplication		
Input-Size	**Scalar**	**Vector**	**Improvement(%)**
128	0.009328	0.002044	4.5636
256	0.039462	0.017946	2.1989
512	0.293343	0.126564	2.3177

- **Note: we calculate the improvement by using the rule = scalar / vector.**

- Vector-Vector Multiplication

- **input-size = 128**

```
yanal@localhost:~$ gcc vector-vector.c -o0
yanal@localhost:~$ ./0
Result:32.000000 Time:0.000001
Result:32.000000 Time:0.000001
yanal@localhost:~$
```

- **Input-size = 256**

```
yanal@localhost:~$ gcc vector-vector.c -o0
yanal@localhost:~$ ./0
Result:60.000000 Time:0.000001
Result:60.000000 Time:0.000001
```

- **Input-size = 512**

```
yanal@localhost:~$ gcc vector-vector.c -o0
yanal@localhost:~$ ./0
Result:121.000000 Time:0.000002
Result:121.000000 Time:0.000001
yanal@localhost:~$
```

- Matrix-Vector Multiplication

- **input-size=128**

```
yanal@localhost:~$ gcc vector-matrix.c -o0
yanal@localhost:~$ ./0
Time:0.000040   Result of summing:4063.000000
Time:0.000015   Result of summing:4063.000000
yanal@localhost:~$
```

- **input-size= 256**

```
yanal@localhost:~$ gcc vector-matrix.c -o0
yanal@localhost:~$ ./0
Time:0.000124   Result of summing:16476.000000
Time:0.000077   Result of summing:16476.000000
yanal@localhost:~$
```

- **input-size= 512**

```
yanal@localhost:~$ gcc vector-matrix.c -o0
yanal@localhost:~$ ./0
Time:0.000518   Result of summing:62793.000000
Time:0.000220   Result of summing:62793.000000
yanal@localhost:~$
```

- Matrix-Matrix Multiplication

- **input-size=128**

```
yanal@localhost:~$ gcc matrix-matrix.c -o0
yanal@localhost:~$ ./0
Time:0.009328   Result of summing:516262.000000
Time:0.002044   Result of summing:516262.000000
yanal@localhost:~$
```

- **input-size= 256**

```
yanal@localhost:~$ gcc matrix-matrix.c -o0
yanal@localhost:~$ ./0
Time:0.039462   Result of summing:4162929.000000
Time:0.017946   Result of summing:4162929.000000
yanal@localhost:~$
```

- **input-size= 512**

```
yanal@localhost:~$ gcc matrix-matrix.c -o0
yanal@localhost:~$ ./0
Time:0.293343   Result of summing:33449564.000000
Time:0.126564   Result of summing:33449564.000000
yanal@localhost:~$
```

Results Explanation:

The SIMD vectorization results indicate consistent improvements across different operations and input sizes. In vector-vector multiplication, The length of time needed to do matrix-matrix multiplication, matrix-vector multiplication, and vector-vector multiplication all tends to grow as the size of the matrices and the size of the vectors do. This is because more operations are needed, so we can see that as size grows, so does time. For instance, when the matrix size is 512, it is larger than 256.

The vectorized approach demonstrates a (1%, 1%, 2%) improvement over the scalar method. For matrix-vector multiplication, the vectorized implementation showcases notable improvements approximately (2.6667%, 1.6104%, 2.3545%). Similarly, in matrix-matrix multiplication, the vectorized approach consistently outperforms the scalar method, with percentage improvements approximately (4.5636%, 2.1989%, 2.3177%). These findings underscore the efficacy of SIMD vectorization in enhancing performance across various computational tasks, highlighting its potential for optimizing mathematical operations in parallel processing environments.

My PC:
CPU generation (i5-12400F), RAM Size: 32.0 GB, Operating System: Linux, vmware workstation 17 player.

Yanal Odeh - 12113273

